Generic Stack and Queue

Set, Stack and Queue in the Java API

Ruhan Alpaydin

Faculty of Computer and Information Sciences
Yeditepe University

v2026-11-05

Outline

1. Three ADTs in One Picture
2. Set in the Java API
3. Stack in the Java API
4. Queue in the Java API
5. Exercises
6. Summary

Three ADTs in One Picture

Set, Stack, Queue

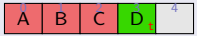
Set

`{apple, banana, cherry}`

no duplicates

Is this element in the set?

Stack (LIFO)



`pop` → **D**

Undo, brackets, backtracking.

Queue (FIFO)

`dequeue` → **A**

Printers, buffers, BFS.

All three are generic types in the Java API: pick the element type in angle brackets.

What “generic” means

- A generic type is parameterized by the type of its elements.
- `Set<String>`, `Stack<Integer>`, `Queue<Person>`.
- The *same class* works for any reference type.

Why generics?

- Type-safe: a wrong type is caught at **compile time**.
- No casting: `names.get(0)` is already a `String`.

ADT	Rule	Java type
Set	no duplicates, no fixed order	`java.util.Set`
Stack	LIFO	`Stack`, `Deque`
Queue	FIFO	`java.util.Queue`

Set in the Java API

What is a Set?

- A Set is a collection of **unique** elements: no duplicates, ever.
- Order is *not* part of the ADT unless the implementation says otherwise.
- Uses: keywords, visited cells, tags, membership tests, deduplication.

Properties

- Does not allow duplicate elements.
- Does not maintain insertion order (for `HashSet`).

Common Set implementations

Class	Ordering	Time	When to use
`HashSet`	none	$O(1)$ avg	default; fast lookup
`TreeSet`	sorted (natural/comparator)	$O(\log n)$	sorted iteration
`LinkedHashSet`	insertion order	$O(1)$ avg	predictable order

- All three implement `java.util.Set`.
- `HashSet` uses `hashCode()` and `equals()`; override both in custom classes.
- `TreeSet` uses `compareTo()` (or a `Comparator`).

Key methods of `Set`

Method	Meaning
`add(E item)`	add; `false` if it was already present
`remove(Object o)`	remove; `true` if found
`contains(Object o)`	membership test
`isEmpty()`	is there no element?
`size()`	number of elements

- `add` returns `boolean`: a free "was it new?" test.
- `contains` is the workhorse: sets exist for this call.

Set: example

codeoutput/SetDemo.java — Java

```java
import java.util.*;

public class SetDemo {
    public static void main(String[] args) {

        Set<String> set = new HashSet<>();
        set.add("Apple");
        set.add("Banana");
        System.out.println(set.add("Apple"));  // false: already
        there
        System.out.println(set);               // [Apple, Banana]
    }
}
```

Terminal — codeoutput/SetDemo.txt

```
$ javac SetDemo.java
$ java SetDemo
false
[Apple, Banana]
```

- The second `add("Apple")` returns `false`.
- `HashSet` does *not* promise any order.
- `LinkedHashSet` gives a predictable one.

Stack in the Java API

Stack: last in, first out

- Insert and remove at the **top**.
- The most recent element leaves first: LIFO.
- Uses: undo, brackets, expression evaluation, backtracking, DFS.



`pop` returns **D**.

Class: java.util.Stack

Method	Meaning
`push(E item)`	add on top
`pop()`	remove and return the top; error if empty
`peek()`	return the top without removing it
`isEmpty()`	is there no element?

- `Stack<E>` is a legacy class (Java 1.0) that extends `Vector`: every call is `synchronized`.
- Modern Java prefers `Deque<E>` (`ArrayDeque`) with the same `push`/`pop`/`peek` names.

Modern alternative: `Deque`

- Same three operations: `push`, `pop`, `peek`.
- `ArrayDeque` is faster than `Stack`, no legacy baggage.
- Rule of thumb: `ArrayDeque` for both stack and queue in new code.

Two refusals

- `ArrayDeque` forbids `null`.
- `pop` on empty throws `NoSuchElementException`.

Stack: example

codeoutput/StackDemo.java — Java

```java
import java.util.*;

public class StackDemo {
    public static void main(String[] args) {
        Stack<String> stack = new Stack<>();
        stack.push("Java");
        stack.push("Python");
        System.out.println(stack.peek());  // Python
        System.out.println(stack.pop());   // Python
        System.out.println(stack.pop());   // Java
    }
}
```

Terminal — codeoutput/StackDemo.txt

```
$ javac StackDemo.java
$ java StackDemo
Python
Python
Java
```

After the two `pop`s the stack is empty: a further `pop` would throw `EmptyStackException`.

Queue in the Java API

Queue: first in, first out

- Insert at the **rear**, remove at the **front**.
- The oldest element leaves first: FIFO.
- Uses: printers, keyboard buffers, scheduling, BFS, message queues.

`dequeue` returns **A**.

java.util.Queue: two forms

- `Queue` is an **interface**, not a class.
- Main implementations: `ArrayDeque`, `LinkedList`.
- Every operation has two forms: one **throws**, one returns a **special value**.

Operation	throws exception	returns special value
insert	`add(e)`	`offer(e)`
remove	`remove()`	`poll()`
examine	`element()`	`peek()`

`isEmpty()` and `size()` complete the interface.

Queue: example

codeoutput/QueueDemo.java — Java

```java
import java.util.*;

public class QueueDemo {
    public static void main(String[] args) {
        Queue<Integer> queue = new ArrayDeque<>();
        queue.offer(10);
        queue.offer(20);
        System.out.println(queue);          // [10, 20]
        System.out.println(queue.peek());   // 10
        System.out.println(queue.poll());   // 10
        System.out.println(queue.poll());   // 20
        System.out.println(queue.poll());   // null
    }
}
```

Terminal — codeoutput/QueueDemo.txt

```
$ javac QueueDemo.java
$ java QueueDemo
[10, 20]
10
10
20
null
```

`ArrayDeque` prints from the **front**; `poll()` on empty returns `null`.

Which Queue class?

Class	Since	Note
`ArrayDeque`	Java 6	recommended: a growing circular array
`LinkedList`	Java 1.2	a doubly linked list
`PriorityQueue`	Java 5	not FIFO: the smallest element leaves first
`ArrayBlockingQueue`	Java 5	bounded; waits when full or empty
`ConcurrentLinkedQueue`	Java 5	unbounded, lock-free, for many threads

- `ArrayDeque` is faster and more compact than `LinkedList`; it forbids `null`.
- `PriorityQueue` is a `Queue` by interface only, *not* by order.

Exercises

Exercise 1: remove duplicates using a Set

Problem

Write a Java program to remove duplicates from a list using a `Set`.

codeoutput/RemoveDup.java — Java

```java
import java.util.*;

public class RemoveDup {
    public static void main(String[] args) {

        String[] words = {"a", "b", "a", "c", "b", "d"};
        HashSet<String> set = new HashSet<>();

        for (int i=0; i<words.length; i++) {
            set.add(words[i]);
        }

        System.out.println("before: " + Arrays.toString(words));
        System.out.println("after:  " + set);
    }
}
```

Terminal — codeoutput/RemoveDup.txt

```
$ javac RemoveDup.java
$ java RemoveDup
before: [a, b, a, c, b, d]
after:  [a, b, c, d]
```

`LinkedHashSet` keeps the order of first occurrence; `HashSet` is faster but loses the order.

Exercise 2: common elements in two lists

Problem

Write a Java program to find common elements between two lists using a `Set`.

codeoutput/CommonElements.java — **Java**

```java
import java.util.*;

public class CommonElements {
    public static void main(String[] args) {
        int[] a = {1, 2, 3, 4, 5};
        int[] b = {3, 4, 5, 6, 7};

    Set<Integer> aSet = new HashSet<>();

    for (int i=0; i<a.length; i++) {
        aSet.add(a[i]);
    }

    Set<Integer> common = new HashSet<>();

    for (int i=0; i<b.length; i++) {
        if (aSet.contains(b[i])) {
            common.add(b[i]);
        }

    }
```

Terminal — codeoutput/CommonElements.txt

```
$ javac CommonElements.java
$ java CommonElements
[3, 4, 5]
```

`retainAll` keeps only the elements also present in the argument: the intersection of the two sets.

Exercise 3: reverse a string using a Stack

Problem

Write a Java program to reverse a string using a `Stack`.

codeoutput/ReverseString.java — **Java**

```java
import java.util.*;

public class ReverseString {
    public static void main(String[] args) {

        String s = "Yeditepe";

        Stack<Character> stack = new Stack<>();

        for (int i=0; i<s.length(); i++) {
             stack.push(s.charAt(i));
        }

        String out = "";

        while (!stack.isEmpty())
              out += stack.pop();

        System.out.println(s + " -> " + out);
    }
}
```

Terminal — codeoutput/ReverseString.txt

```
$ javac ReverseString.java
$ java ReverseString
Yeditepe -> epetideY
```

Push every character, then pop them: the last pushed is the first to come out, so the string is reversed.

Exercise 4: queue from two stacks

Problem

Write a Java program to implement a queue using two stacks.

codeoutput/QueueTwoStacks.java **Java**

```java
import java.util.*;
public class QueueTwoStacks<T> {
    private final Stack<T> in  = newStack<>();
    private final Stack<T> out = new Stack<>();
    public void enqueue(T x) { in.push(x); }
    public T dequeue() {
        if (out.isEmpty())
            while (!in.isEmpty()) out.push(in.pop());
        return out.pop();
    }
    public boolean isEmpty() { return in.isEmpty() && out.isEmpty();}
    public static void main(String[] args) {
        QueueTwoStacks<String> q = new QueueTwoStacks<>();
        q.enqueue("A"); q.enqueue("B"); q.enqueue("C");
        System.out.println(q.dequeue());   // A
        System.out.println(q.dequeue());   // B
        q.enqueue("D");
        System.out.println(q.dequeue());   // C
        System.out.println(q.dequeue());   // D
        System.out.println(q.isEmpty());   // true
    }
}
```

Terminal codeoutput/QueueTwoStacks.txt

```
$ javac QueueTwoStacks.java
$ java QueueTwoStacks
A
B
C
D
true
```

Two reversals compose to the original order: `in` receives, `out` serves; when `out` is empty, transfer everything.

Summary

Big ideas

- **Set**: unique elements. `HashSet` (unordered), `TreeSet` (sorted), `LinkedHashSet` (insertion order).
- **Stack**: LIFO. `java.util.Stack` is legacy; prefer `Deque` (`ArrayDeque`) with `push`/`pop`/`peek`.
- **Queue**: FIFO. Interface `Queue`; `ArrayDeque` or `LinkedList`. `add`/`remove`/`element` throw, `offer`/`poll`/`peek` return special values.
- `PriorityQueue` implements `Queue`, but is *not* FIFO.
- The API classes are generic: one class, any element type.

Check yourself

1. What does `Set.add` return? Why is that useful?
2. `HashSet` vs `TreeSet` vs `LinkedHashSet`: what is the iteration order in each?
3. `Stack.pop` on an empty stack: what happens? What about `Queue.poll`?
4. Why is `ArrayDeque` preferred over `Stack` in new code?
5. Which `Queue` methods throw? Which return a special value?
6. Why is `PriorityQueue` not FIFO?
7. Why does a queue made of two stacks behave like a FIFO?

References

1. Oracle Java Documentation.
2. GeeksforGeeks.
3. Joshua Bloch, *Effective Java* (best practices for the Java collections framework).
4. Herbert Schildt, *Java: The Complete Reference*.
5. `https://docs.oracle.com/javase/8/docs/api/`
6. `https://docs.oracle.com/javase/tutorial/collections/`

Questions?